

Method Overloading Resolution Programs :

byte -> short -> int -> long -> float -> double

char -> int -> long -> float -> double

MOLR1.java

```
-----
void main()
{
    accept(6);
}

public void accept(double d)
{
    IO.println("double");
}
public void accept(float d)
{
    IO.println("float");
}
```

Note: Here float will be executed because float is the most specific type.

MOLR2.java

```
-----
void main()
{
    accept(6);
}

public void accept(int d)
{
    IO.println("int");
}
public void accept(char d)
{
    IO.println("char");
}
```

Here 6 is int type so int will be executed using exact match Rule.

MOLR3.java

```
-----
void main()
{
    accept();
}

public void accept(int ...d)
{
    IO.println("int");
}
public void accept(char ...d)
{
    IO.println("char");
}
```

char will be executed becoz char is more specific type.

MOLR4.java

```
-----
void main()
{
    accept();
}

public void accept(short ...d)
{
    IO.println("short");
}
public void accept(char ...d)
{
    IO.println("char");
}
```

Here we will get compilation error because there is no relation between char and short based on the specific type rule.

MOLR5.java

```
-----
void main()
{
    accept();
}

public void accept(short ...d)
{
    IO.println("short");
}
public void accept(byte ...d)
{
    IO.println("byte");
}
```

Here byte will be executed because byte is the specific type.

MOLR6.java

```
-----
void main()
{
    accept();
}

public void accept(double ...d)
{
    IO.println("double");
}
public void accept(long ...d)
{
    IO.println("long");
}
```

Here long will be executed because long is the most specific type.

MOLR7.java

```
-----
void main()
{
    accept(9);
}

public void accept(int d)
{
    IO.println("int");
}
public void accept(long s)
{
    IO.println("long");
}
```

Note : Here int will be executed because int is the exact match

MOLR8.java

```
-----
void main()
{
    accept(9);
}

public void accept(long s)
{
    IO.println("Widening");
}
public void accept(Integer i)
{
    IO.println("Autoboxing");
}
```

Here widening is having more priority

MOLR9.java

```
-----
void main()
{
    accept(9);
}

public void accept(int ...s)
{
    IO.println("Var args");
}
public void accept(Integer i)
{
    IO.println("Autoboxing");
}
```

Here Autoboxing will be executed.

MORL10.java

```
-----
void main()
{
    accept(9);
}

public void accept(int ...s)
{
    IO.println("Var args");
}
public void accept(int i)
{
    IO.println("int");
}
```

Here int will be executed

MORL11.java

```
-----
void main()
{
    accept(9);
}

public void accept(int ...s)
{
    IO.println("Var args");
}
public void accept(int []i)
{
    IO.println("int array");
}
```

Here we will get compilation error Internally var args is an array

MORL12.java

```
-----
void main()
{
    accept(9,2);
}

public void accept(int ...s)
{
    IO.println("Var args");
}
public void accept(long x , long y)
{
    IO.println("Widening");
}
```

Here Widening will be executed.

MORL13.java

```
-----
void main()
{
    accept(9,2);
}

public void accept(int x, long y)
{
    IO.println("int - long");
}
public void accept(long x , int y)
{
    IO.println("long - int");
}
```

Compilation error, reference is ambiguous

MOL14.java

```
-----
void main()
{
    accept(9);
}

public void accept(Double y)
{
    IO.println("Double");
}
public void accept(Long x)
{
    IO.println("Long ");
}
```

Note : Compilation error

What is ECLIPSE IDE :

IDE stands for "**Integrated Development Environment**". By using eclipse IDE, we can develop, compile and execute our java program in a single window.

The main purpose of Eclipse IDE to reduce the **development time**, once the development time will be reduced then automatically the cost of the project will be reduced

How to create a Project in Eclipse IDE:

File -> new -> Project OR Java Project -> Provide the name for the project (Batch 59)-> Finish

What is a Package in java:

A package is nothing but folder in windows. It is used to arrange the classes and interfaces into a particular group.

If we arrange our java classes into a particular group by using packages (folders) then we will get the following two advantages:

- 1) Fast searching is possible.
- 2) Name can be reusable.

Types of Packages:

1) Predefined OR Built-in package: The packages which are created by java software people for arranging the programs are called predefined package.

Example : java.lang, java.util, java.io, java.sql, java.net and so on

2) Userdefined Package OR Custom package : The packages which are created by user for arranging the user-defined programs are called user-defined package.

Example :

```
basic;
com.ravi.basic;
com.tcs.online_shopping;
```